Доклад

Понятие процесса, виды процессов и его состояния в операционных системах

Юсупова Амина Руслановна

Оглавление

1 Введение 1
2 Основная часть 2
2.1 Определение процесса 2
2.2 Классификация процессов 4
2.3 Программные потоки 5
3 Заключение 7
4 Список литературы 7

1 Введение

Современные операционные системы представляют собой сложные программные комплексы, обеспечивающие эффективное управление ресурсами вычислительной системы и предоставляющие пользователям и приложениям удобный интерфейс для работы. Одно из главных понятий, с которыми работают компьютеры, - это процесс.

Процесс - это выполняемая программа вместе с текущими значениями счетчика команд, регистров и переменных. Именно понятие процесса позволяет операционной системе создавать иллюзию одновременного выполнения множества задач на одном процессоре, обеспечивая многозадачность

Понимание того, что такое процесс, какими свойствами и состояниями он обладает, а также какие виды процессов существуют, является ключевым для изучения принципов работы операционных систем. Без этого знания невозможно понять, как планируются задачи, как предотвращаются ошибки при доступе к ресурсам

В данном докладе будут рассмотрены основные определения и характеристики процессов, классификация их видов, а также подробно разобрана модель состояний процесса, отражающая динамику его выполнения в системе от создания до завершения.

Проблема: Несмотря на то, что процесс является фундаментальным понятием операционной системы, существует разрыв между теоретическим определением процесса и

пониманием механизмов его функционирования в реальной многозадачной среде.

Цель работы: раскрыть понятие процесса как одно из ключевых понятий операционных систем, рассмотреть классификацию процессов и их жизненный цикл, а также показать, каким образом операционная система управляет их выполнением.

Задачи:

1. Дать определение процессу и объяснить, чем он отличается от программы.
2. Перечислить и кратко охарактеризовать основные виды процессов
3. Описать состояния процесса и их смену.

Ход работы:

1. Поиск и анализ теоретического материала
2. Структурирование информации и выделение ключевых разделов
3. Углубленное изучение темы программных потоков
4. Подготовка текста доклада и презентации

Гипотеза: вероятно, глубокое понимание концепции процесса, его классификации и особенно модели состояний (жизненного цикла) является необходимым условием для осознания принципов функционирования операционной системы в целом.

2 Основная часть

2.1 Определение процесса

Концепция процесса является основополагающей для любой операционной системы. Она предполагает два аспекта: во-первых, процесс является носителем данных и, во-вторых, он собственно и выполняет операции, связанные с обработкой этих данных.

В чём же основное отличие программы и процесса?

Программа — это пассивный объект (файл на диске), а процесс — активный (выполняющийся экземпляр программы)

В качестве примеров процессов можно назвать прикладные программы пользователей, утилиты и другие системные обрабатываемые программы. Процессом может

быть редактирование какого-либо текста, трансляция исходной программы, ее компоновка, исполнение. Причем, трансляция какой-нибудь исходной программы является одним процессом, а трансляция следующей исходной программы - другим процессом, поскольку транслятор как объединение программных модулей здесь выступает как одна и та же программа, но данные, которые он обрабатывает, являются разными.

При исполнении программ на процессоре чаще всего различают следующие характерные отдельные состояния:

- **порождение** – подготовка для первого исполнения на процессоре;
- **активное состояние**, или состояние “Счет” – программа исполняется на процессоре;
- **ожидание** – программа не исполняется на процессоре по причине занятости какого-либо требуемого ресурса;
- **готовность** – программа не исполняется, но для исполнения предоставлены все необходимые в текущий момент ресурсы, кроме центрального процессора;
- **окончание** – нормальное или аварийное окончание исполнения программы, после которого процессор и другие ресурсы ей не предоставляются.

Процесс находится в каждом из своих допустимых состояний в течение некоторого времени, после чего переходит в какое-то другое допустимое состояние. Состав допустимых состояний, а также допустимые переходы из состояния в состояние обычно задают в форме графа существования процесса.

Для ОС процесс в такой трактовке рассматривается как объект, в отношении которого требуется обеспечить реализацию каждого из допустимых состояний, а также допустимые переходы из состояния в состояние в ответ на события, которые могут явиться причиной таких переходов. Эти события могут инициироваться и самими процессами, которые способны затребовать процессор или какой-либо другой ресурс, необходимый для исполнения программы.

2.2 Классификация процессов

Процессы определяются рядом временных характеристик. В некоторый момент времени процесс может быть порожден (образован), а через некоторое время закончен. Интервал между этими моментами называют **интервалом существования процесса**.

В момент порождения последовательность и длительность пребывания процесса в каждом из своих состояний (**трасса процесса**) в общем случае непредсказуемы. Следовательно, непредсказуема и длительность интервала существования. Однако отдельные виды процессов требуют такого планирования, чтобы гарантировать окончание процесса до наступления некоторого конкретного момента времени. Процессы такого класса называют **процессами реального времени**. В другой класс входят процессы, время существования которых должно быть не более интервала времени допустимой реакции компьютера на запросы пользователя. Процессы такого класса называют **интерактивными**. Процессы, не вошедшие в эти классы, называют **пакетными**.

В любой ОС по требованию существующего или существовавшего процесса проводится работа по порождению процессов. Процесс, задающий данное требование, называют **порождающим**, а создаваемый по требованию – **порожденным**. Если порожденный процесс на интервале своего существования в свою очередь выдает требование на порождение второго процесса, то он одновременно становится и порождающим.

Если интервалы двух процессов не пересекаются во времени, то такие два процесса называют **последовательными** друга относительно друга. Если на рассматриваемом интервале времени существуют одновременно два процесса, то они на этом интервале являются **параллельными** друг относительно друга. Если на рассматриваемом интервале найдется хотя бы одна точка, в которой существует один процесс, но не существует другой, и хотя бы одна точка, в которой оба процесса существуют одновременно, то такие два процесса называют **комбинированными**.

Программные процессы принято делить на **системные** и **пользовательские**. При развитии системного процесса исполняется программный код из состава операционной системы. При развитии пользовательского процесса исполняется пользовательская (прикладная) программа.

Процессы независимо от их вида могут быть **взаимосвязанными** или **изолированными** друг от друга. Два процесса являются взаимосвязанными, если между ними поддерживаются с помощью системы управления процессами какого-либо рода связи: функциональные, пространственно-временные, управляющие, информационные и т.д. В противном случае они являются изолированными (точнее – процессами со слабыми связями, так как при отсутствии явных связей они могут быть связаны косвенно, и определенным образом влиять на развитие друг друга). Когда необходимо подчеркнуть связь между взаимосвязанными процессами по ресурсам, их называют **конкурирующими**.

Управление взаимосвязанными процессами в составе ОС основано на контроле и удовлетворении определенных ограничений, которые накладываются на порядок выполнения таких процессов. Данные ограничения определяют **виды отношений**, которые допустимы между процессами, и составляют в совокупности синхронизирующие правила.

Отношение предшествования. Для двух процессов это отношение означает, что первый процесс должен переходить в активное состояние всегда раньше второго.

Отношение приоритетности. Процесс с приоритетом Р может быть переведен в активное состояние только при соблюдении двух условий: в состоянии готовности к рассматриваемому процессору нет процессов с большим приоритетом; процессор либо свободен, либо используется процессом с меньшим, чем Р, приоритетом.

Отношение взаимного исключения. Здесь два процесса используют общий ресурс. При этом совокупность действий над этим ресурсом в составе одного процесса называют критической областью. Критическая область одного процесса не должна выполняться одновременно с критической областью над этим же ресурсом в составе другого процесса [1].

2.3 Программные потоки

В традиционных операционных системах у каждого процесса есть адресное пространство и один поток управления. Тем не менее, нередко возникают ситуации, в которых желательно иметь несколько потоков управления, квазипараллельно выполняющихся в одном адресном пространстве так, как будто они представляют собой отдельные процессы (за исключением общности адресного пространства). Такие потоки управления называются **программными потоками**.

Программные потоки - мощный инструмент, когда приложению необходимо выполнять сразу несколько действий (по крайней мере, они создают иллюзию одновременного выполнения нескольких операций). Если задачу приложения можно разделить на различные блоки: ввод или вывод, связь, обработка некоторых особых событий и т.д., то потоки могут быть органично встроены в программные решения. Сделав приложение многопоточным, можно получить дополнительные возможности управления с помощью изменения приоритетов потоков или, например, снять большую нагрузку на приложение, распределив ее между потоками.

Существует однопоточность и многопоточность.

Однопоточность - это режим, при котором программа может обрабатывать только одну задачу за раз. Система в одном потоке работает со всеми задачами, выполняя их поочерёдно.

Многопоточность — это выполнение нескольких потоков внутри одного процесса. Все потоки принадлежат одной программе и могут совместно использовать ее память и ресурсы.

Многопоточность имеет большие преимущества:

- **Увеличение скорости** (по сравнению с использованием обычных процессов). Многопоточность основана на использовании **облегченных процессов** ,работающих в общем пространстве виртуальной памяти. Благодаря многопоточности, не возникает больше неэффективных ситуаций, типичных для классической системы UNIX, в которой каждая команда shell (даже команда вывода содержимого *текущей директории* **ls** исполнялась как **отдельный процесс**, причем в своем собственном адресном пространстве. В противоположность облегченным процессам, обычные процессы (имеющие собственное адресное пространство) часто называют **тяжеловесными.**
- **Использование общих ресурсов**. Потоки одного процесса используют *общую память* и файлы.
- **Экономия**. Благодаря многопоточности, достигается значительная экономия памяти, по причинам, объясненным выше. Также достигается и экономия времени, так как *переключение контекста* на облегченный процесс, для которого требуется только сменить стек и восстановить значения регистров, значительно быстрее, чем на обычный процесс [2].

3 Заключение

Подводя итог данному докладу, можно с уверенностью утверждать, что понятие процесса является важным, на котором строится архитектура любой современной операционной системы. Проведенный анализ позволил достичь поставленной цели и решить все задачи.

Во-первых, было выявлено основное различие между пассивной программой (набором инструкций на диске) и активным процессом (динамическим объектом, обладающим контекстом и потребляющим ресурсы). Это различие позволяет ОС управлять выполнением задач, а не просто хранить код.

Во-вторых, рассмотрение видов процессов показало, насколько сложной иерархической структурой обладает система управления задачами. Деление на системные и пользовательские, пакетные и интерактивные, порождающие и порожденные процессы, а также наличие различных отношений между ними (предшествование, приоритетность, взаимное исключение) демонстрирует гибкость и адаптивность ОС под разные требования - от обслуживания одного пользователя до работы в жестких временных рамках систем реального времени.

В-третьих, анализ модели состояний процесса (Создание - Готовность - Исполнение - Ожидание - Завершение) позволил понять динамику его существования. Переключение между этими состояниями - это тот самый механизм, который лежит в основе многозадачности.

Кроме того, введение понятия программных потоков (нитей) как «облегченных» процессов внутри одного адресного пространства позволило показать пути дальнейшего развития концепции многозадачности. Многопоточность, позволяя эффективно использовать общие ресурсы и ускорять вычисления, наглядно демонстрирует, как эволюционируют идеи управления процессами для достижения максимальной производительности.

Таким образом, понимание процессов, их свойств, классификации и жизненного цикла формирует необходимую теоретическую базу для дальнейшего изучения планирования задач, синхронизации и оптимизации работы операционных систем.

4 Список литературы

1. https://studfile.net/preview/2495676/page:3/
2. https://intuit.ru/studies/courses/641/497/lecture/11284?page=2